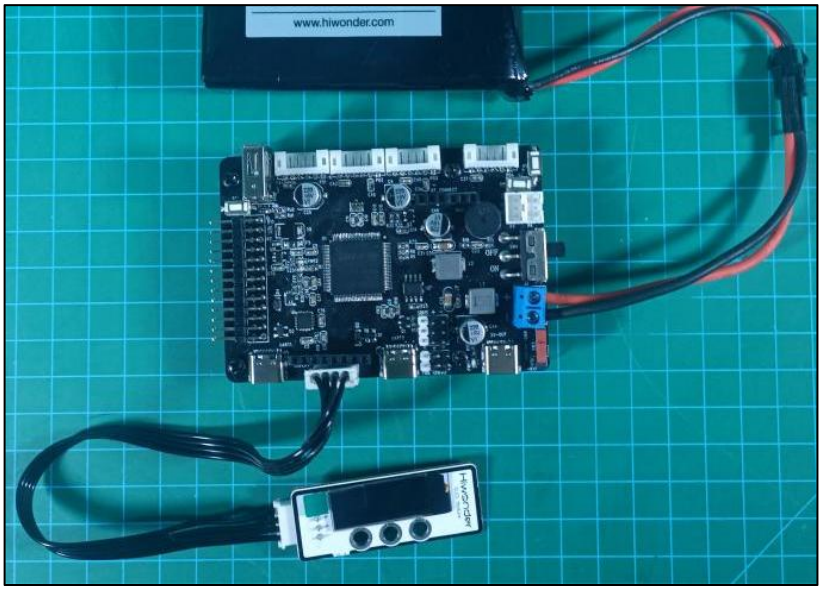
Lesson 5 RRC QMI8658 Accelerometer and Gyroscope Source Code Analysis

1. Program Description

This chapter focuses on reading data from the built-in MPU6050 on the STM32 development board, calculating the current attitude of the board, and outputting the data via the serial port and displaying it on the OLED.

2. Hardware Connection

Connect the OLED display to the main control board via I2C expansion interface. The hardware connection diagram is as follows:



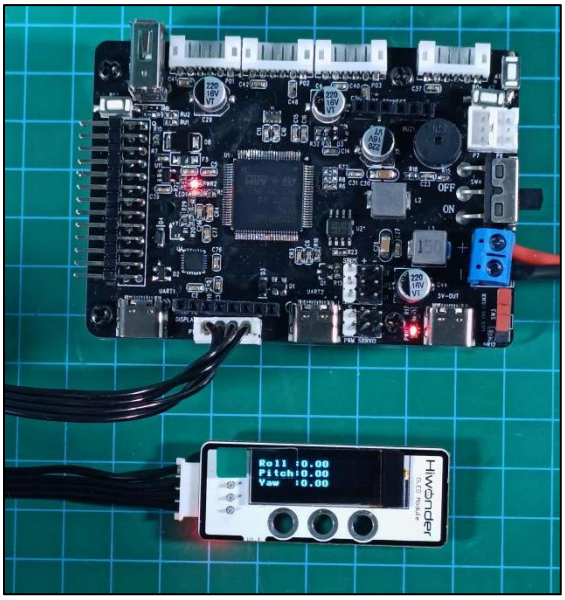
3. Program Download

The example for this section is "**rosrobotcontrollerm4_IMU**".

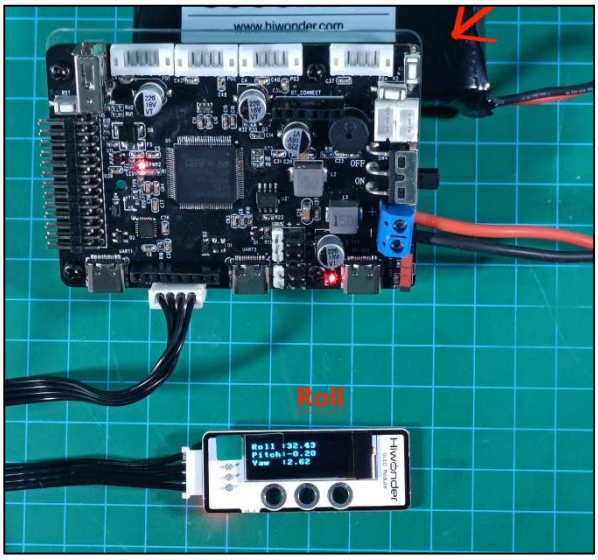
4. Program Outcome

After downloading the program, press the **RESET** button to see the real-time attitude data of the development board displayed on the OLED screen.

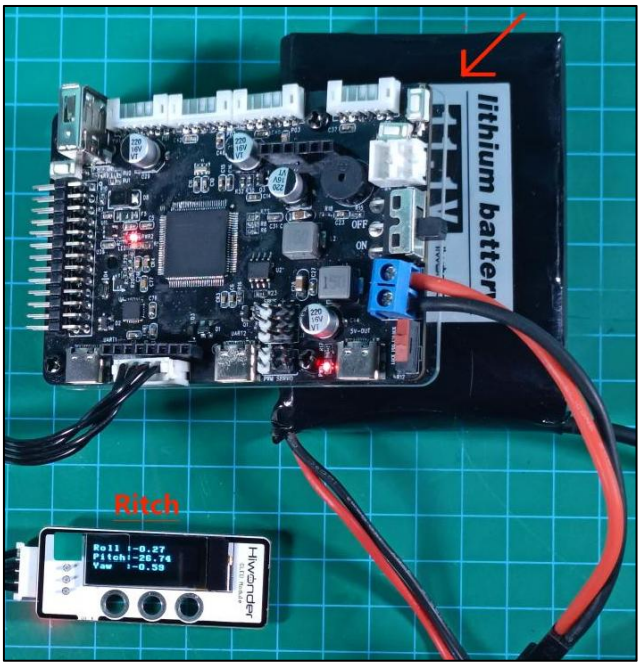
After reset, the data displayed for the three axes is basically zero (with some minor errors), as shown below:



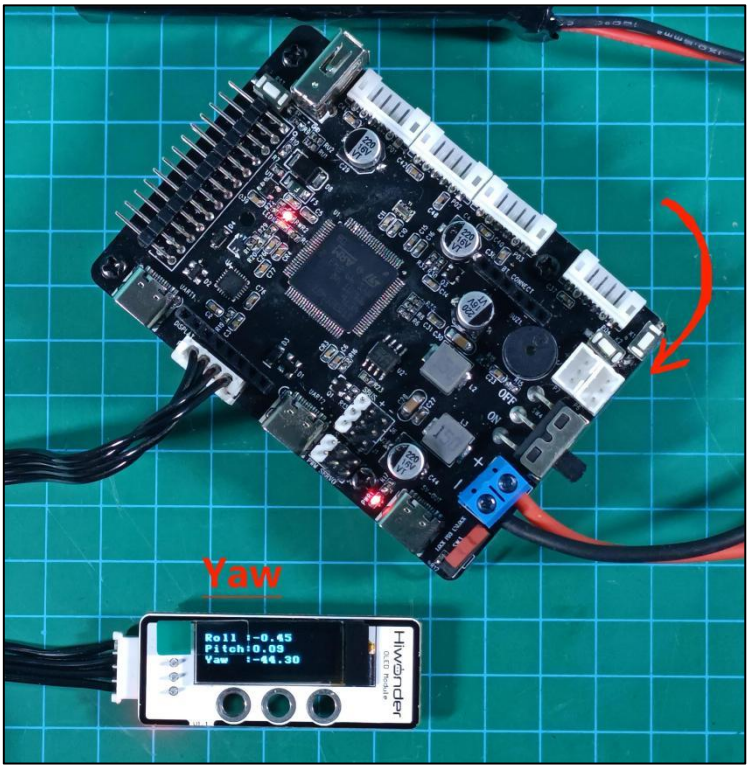
For forward and backward tilting, mainly the Roll data changes.



For left and right tilting, mainly the Pitch data changes.



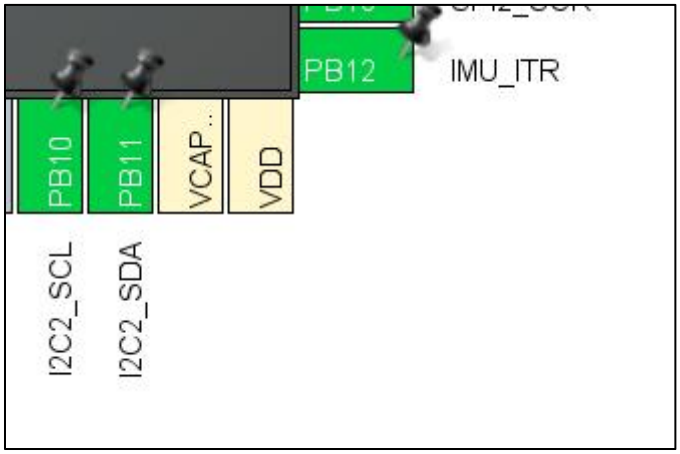
For plane rotation, mainly the Yaw data changes.



5. Pin Configuration

View the pin information through stm32cubemx software. The MCU communicates with the IMU using I2C, and the IMU sends interrupt signals for the MCU to collect IMU data.

The corresponding I2C chip pins are PB10 and PB11; the interrupt pin is PB12.



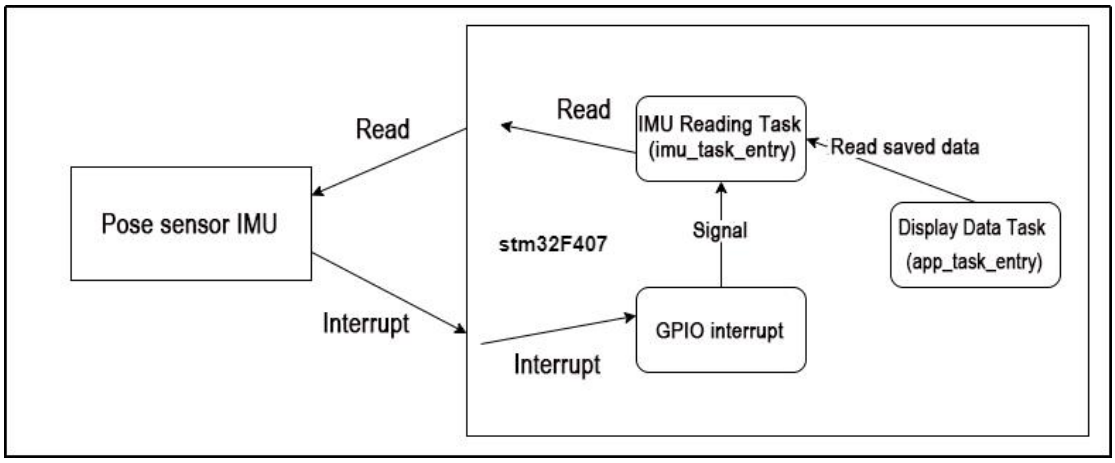
6. Source Code Analysis

The source code for this section is located in the "Appendix/Source Code" folder. The main program framework is designed as follows:

Analyze interrupts through the Application/User/Core/stm32f4xx_it.c file

Analyze the display task through the Hiwonder/System/app.c file

Analyze the IMU reading task through the Hiwonder/Porting/imu_porting.c file



The analysis is divided into three parts:

1. External interrupt handling
2. Loop reading task
3. Loop display task

6.1 External Interrupt

In the Application/User/Core/stm32f4xx_it.c file, the `EXTI15_10_IRQHandler()` function handles the external interrupt. Before IMU module provides a new set of data for host computer to read, it sends a signal (a voltage level change) through one of its INT pins to trigger the interrupt.

The main controller enables external interrupts on the pin connected to the IMU to detect voltage level changes, thereby monitoring the IMU data transmission status. When an external interrupt is triggered, it unblocks the flag indicating IMU data readiness (i.e., the semaphore IMU_data_readyHandle in the RTOS), as shown below:

```
/**
  * @brief This function handles EXTI line[15:10] interrupts.
  */
void EXTI15_10_IRQHandler(void)
{
  /* USER CODE BEGIN EXTI15_10_IRQn 0 */
    extern osSemaphoreId_t IMU_data_readyHandle;
    if(__HAL_GPIO_EXTI_GET_IT(IMU_ITR_Pin) != RESET) {  //是否为IMU中断引脚信号
        __HAL_GPIO_EXTI_CLEAR_IT(IMU_ITR_Pin);  //清除IMU中断引脚信号
    //发送信号量
        osSemaphoreRelease(IMU_data_readyHandle);
    }
  /* USER CODE END EXTI15_10_IRQn 0 */
  /* USER CODE BEGIN EXTI15_10_IRQn 1 */

  /* USER CODE END EXTI15_10_IRQn 1 */
}
```

6.2 IMU Reading Task

The reading task first calls `begin()` to initialize the IMU and its parameters. After that, it enters a loop to continuously wait for and read IMU data.

```
void imu_task_entry(void *argument)
{

  //声明IMU的外部句柄
    extern osSemaphoreId_t IMU_data_readyHandle;
    extern osMutexId_t oled_mutexHandle;
  //声明存储姿态数据数组
    extern float rpy[3];

    if(begin() == 0)
    {
        printf("qmi8658_init fail");
    }else{
        printf("qmi8658_init success");
  }
    osDelay(100);
    PacketReportIMU_Raw_TypeDef report;

    for(;;) {
    //等待IMU中断发送的信号量，若没有，则一直阻塞等待 osWaitForever
    osSemaphoreAcquire(IMU_data_readyHandle, osWaitForever);

    //获取欧拉角
    GetEulerAngles(&rpy[0],&rpy[1],&rpy[2]);
```

The `osSemaphoreAcquire()` function in the RTOS system acquires the semaphore:

- Parameter 1: Semaphore handle (here, the IMU semaphore handle)
- Parameter 2: Timeout duration (ms), with osWaitForever meaning to wait indefinitely.

If there is no semaphore in the queue, the function waits and executes other tasks. When a semaphore is available, it retrieves it and continues running the subsequent program.

The `GetEulerAngles()` function is called to read the IMU's Euler angle data, which is then stored into the rpy array.

6.3 IMU Data Display Task

This task prints the data stored in the IMU object to the serial port and displays it on the OLED. The program flow is as follows:

Initialize the OLED;

Loop:

The real-time Euler angle data stored in the `rpy` array is displayed on the OLED screen and also printed to the serial port using `printf`.

Delay for 150ms；

```
void app_task_entry(void *argument)
{
  //OLED和u8g2库初始化
  u8g2_init();

  //定义存储姿态数据数组

  //定义一个字符数组
  static char msg[16];

  //设置u8g2的字体显示
  u8g2_SetFontMode(u8g2, 1);
  u8g2_SetFontDirection(u8g2, 0);
  u8g2_SetFont(u8g2, u8g2_font_victoriabold8_8r);

    for(;;) {

      //清除OLED缓冲区
      u8g2_ClearBuffer(u8g2);

      //rpy[0]存储的是Roll的数据
      //将浮点数转换成字符串，并储存到msg数组里
      sprintf(msg, "Pitch :%3.2f", FusionRadiansToDegrees(rpy[0]));
      //串口打印该数据
      printf("%s\n",msg);
      //将该数据写入OLED缓冲区的(0,9)位置开始区域          1
      u8g2_DrawStr(u8g2, 0, 9, msg);

      //rpy[1]存储的是Pitch的数据
      //将浮点数转换成字符串，并储存到msg数组里
      sprintf(msg, "Roll :%3.2f", FusionRadiansToDegrees(rpy[1]));
      //串口打印该数据
      printf("%s\n",msg);
      //将该数据写入OLED缓冲区的(0,20)位置开始区域
      u8g2_DrawStr(u8g2, 0, 20, msg);

      //rpy[2]存储的是Yaw的数据
      //将浮点数转换成字符串，并储存到msg数组里
      sprintf(msg, "Yaw :%3.2f", FusionRadiansToDegrees(rpy[2]));
      //串口打印该数据
      printf("%s\n",msg);
      //将该数据写入OLED缓冲区的(0,31)位置开始区域
      u8g2_DrawStr(u8g2, 0, 31, msg);

      //将OLED缓冲区的数据发送至OLED模块，使其刷新屏幕
      u8g2_SendBuffer(u8g2);

      //延时 150 ms
      osDelay(150);
    }
}
```

As described above, in the "**IMU reading task**," each time the semaphore changes, the real-time IMU data is immediately read and stored into the global rpy array. Therefore, in the current task, it is sufficient to continuously output the three elements of the rpy array (Euler angles) via the OLED display and serial port to represent the real-time attitude data.

Since refreshing the OLED screen takes time, a delay is required in each loop cycle to ensure proper display of the current data and smooth screen updates.